

Spam Classification on the Spambase Dataset: Naive Bayes and k -Nearest Neighbours

NATHANIEL CHRISTIAN
Reg. No. 3122247001037

August 1, 2026

1 Aim and Objective

The aim of this experiment is to build, tune, and compare multiple classifiers for detecting spam email using the UCI Spambase dataset. Specifically, the objectives are to:

- Perform exploratory data analysis (EDA) on the Spambase dataset, including cleaning, deduplication, and outlier handling.
- Train and evaluate four classifiers – Gaussian Naive Bayes, Multinomial Naive Bayes, Bernoulli Naive Bayes, and k -Nearest Neighbours (KNN) – on the cleaned dataset.
- Tune the KNN hyperparameters (k , weighting scheme, and search algorithm) using both exhaustive (GridSearchCV) and randomized (RandomizedSearchCV) search, and compare their efficiency.
- Compare classifiers using Accuracy, Precision, Recall, F1-score, ROC-AUC, and 5-fold cross-validation, rather than relying on a single metric or a single train/test split.
- Analyze the computational complexity and measured training/ prediction time of each model, including a direct comparison of the KDTree and BallTree nearest-neighbour search structures.
- Draw conclusions about which model(s) are best suited to spam detection given the trade-offs between precision, recall, and speed.

2 Theory

2.1 Naive Bayes Classifiers

Naive Bayes classifiers apply Bayes' theorem with the (naive) assumption that features are conditionally independent given the class label:

$$P(y | x_1, \dots, x_d) \propto P(y) \prod_{i=1}^d P(x_i | y) \quad (1)$$

The three variants used here differ only in how $P(x_i | y)$ is modelled:

- **Gaussian NB** assumes each feature is normally distributed within each class, i.e. $P(x_i | y) \sim \mathcal{N}(\mu_{i,y}, \sigma_{i,y}^2)$. This is a poor fit for zero-inflated, right-skewed features such as word frequencies.
- **Multinomial NB** models $P(x_i | y)$ as a multinomial distribution, which is naturally suited to discrete counts or frequency-style features – close to how the Spambase word/character frequency features are actually generated.
- **Bernoulli NB** assumes each feature is binary (present/absent), binarizing continuous features at a threshold (default 0.0) before applying a Bernoulli likelihood.

2.2 k -Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based (“lazy”) learner. It stores the entire training set and, for prediction, finds the k closest training points to a query point (by a distance metric, typically Euclidean) and assigns the majority class among them, optionally weighted by inverse distance (weights= $1/\text{distance}$). Because no model is fit during training, all computational cost is deferred to prediction time – unless an indexing structure (KDTree, BallTree) is used to accelerate the neighbour search.

2.3 Feature Scaling

All features were scaled with `MinMaxScaler` to $[0, 1]$ prior to modelling, since KNN's distance calculations and `BernoulliNB`'s binarization are both sensitive to feature scale.

2.4 Hyperparameter Search

`GridSearchCV` exhaustively evaluates every combination in a hyperparameter grid via cross-validation, guaranteeing the best combination within the grid but scaling combinatorially with grid size. `RandomizedSearchCV` instead samples a fixed number of random combinations, trading an exhaustive guarantee for substantially lower computational cost – often reaching a comparable optimum in a fraction of the time.

2.5 Evaluation Metrics

- **Accuracy:** fraction of correct predictions overall.
- **Precision:** of predicted spam, the fraction that is actually spam.
- **Recall:** of actual spam, the fraction correctly identified.
- **F1-score:** harmonic mean of precision and recall.
- **ROC-AUC:** area under the ROC curve, measuring discriminative ability across all classification thresholds.

Because the dataset is class-imbalanced after cleaning (roughly 76:24 non-spam:spam), accuracy alone can be misleading, and the other metrics – especially ROC-AUC and F1 – are weighted more heavily in the comparison.

3 Dataset

The **Spambase** dataset (UCI Machine Learning Repository) contains 4601 emails, each described by 57 numeric features plus a binary class label (1 = spam, 0 = non-spam):

- **48 word_freq * features:** percentage of words in the email matching a specific word (e.g. free, money, business).
- **6 char_freq * features:** percentage of characters matching a specific symbol (e.g. !, \$).
- **3 capital run length * features:** average, longest, and total length of uninterrupted sequences of capital letters.
- **class:** target label (1 = spam, 0 = non-spam).

All 57 predictor columns are numeric (float64 or int64), with no missing values in the raw data.

4 Exploratory Data Analysis (EDA)

4.1 Console Output

```
~/ML-EXP1 >> python3 exp2.py
```

```
=====
```

```
EXPLORATORY DATA ANALYSIS
```

```
=====
```

First 5 Rows

	word_freq_make	word_freq_address	...	capital_run_length_total	class
0	0.00	0.64	...	278	1
1	0.21	0.28	...	1028	1
2	0.06	0.00	...	2259	1
3	0.00	0.00	...	191	1
4	0.00	0.00	...	191	1

[5 rows x 58 columns]

Dataset Shape: (4601, 58)

Missing Values Before Cleaning: 0 (all 58 columns)

Missing Values After Cleaning: 0 (all 58 columns)

Duplicate Rows: 391

Shape After Removing Duplicates: (4210, 58)

Numerical Statistics (selected)

	word_freq_make	word_freq_address	...	capital_run_length_total	class
count	4210.000000	4210.000000	...	4210.000000	4210.000000
mean	0.104366	0.112656	...	291.181948	0.398812
std	0.300005	0.454260	...	618.654838	0.489712
min	0.000000	0.000000	...	1.000000	0.000000
25%	0.000000	0.000000	...	40.000000	0.000000
50%	0.000000	0.000000	...	101.500000	0.000000
75%	0.000000	0.000000	...	273.750000	1.000000
max	4.540000	14.280000	...	15841.000000	1.000000

[8 rows x 58 columns]

Removing Outliers (IQR Method)

word_freq_make: skipped (IQR is 0, likely a sparse/constant column)

word_freq_all: 293 outliers flagged

word_freq_our: 449 outliers flagged

word_freq_mail: 748 outliers flagged

word_freq_will: 250 outliers flagged

word_freq_free: 830 outliers flagged

word_freq_you: 58 outliers flagged

word_freq_your: 193 outliers flagged

word_freq_hp: 1012 outliers flagged

word_freq_re: 805 outliers flagged

char_freq_%28: 255 outliers flagged

char_freq_%21: 373 outliers flagged

char_freq_%24: 730 outliers flagged

capital_run_length_average: 329 outliers flagged

capital_run_length_longest: 418 outliers flagged

capital_run_length_total: 497 outliers flagged

(remaining word_freq_* columns skipped: IQR = 0, sparse/constant)

Shape Before Outlier Removal: (4210, 58)

Shape After Outlier Removal: (809, 58)

4.2 Summary Table: Cleaning Stages

4.3 Discussion

The correlation matrix (Figure 1) shows that most feature pairs have near-zero correlation, indicating that multicollinearity is not a major concern for this dataset and most features can be retained.

The numeric feature distributions (Figure 2) reveal that nearly all word_freq_* columns are heavily zero-inflated and right-skewed: most individual words do not appear in most emails, so values are 0 for the majority of rows. This explains why many columns were skipped during

Table 1: Dataset shape at each cleaning stage

Stage	Rows	Columns
Original	4601	58
After missing value removal	4601	58
After duplicate removal	4210	58
After IQR outlier removal	809	58

IQR-based outlier removal ($Q1 = Q3 = 0$ for those columns).

The class distribution (Figure 3) shows a moderate class imbalance after cleaning: 616 non-spam versus 193 spam emails (roughly 76:24). This suggests that accuracy alone would be a misleading evaluation metric for a downstream spam classifier; precision, recall, F1-score, or ROC-AUC are more appropriate, along with possible class-weighting or resampling during model training.

Outlier removal reduced the dataset from 4210 rows (after deduplication) to 809 rows, an 81% reduction, driven by the compounding effect of applying IQR filtering across 34 skewed columns simultaneously. A milder filtering approach (e.g. a wider IQR multiplier, or skipping outlier removal on skewed, zero-inflated count-like features) would likely preserve more data for downstream classification.

5 Code

The full training and evaluation pipeline is given below. It performs EDA, a leakage-free train/test split and scaling, trains all four models, evaluates them, tunes KNN hyperparameters via both grid and randomized search, compares KDTree vs. BallTree, runs 5-fold cross-validation, and produces a single combined results dashboard (Section 9).

```

1 from eda_function import perform_eda
2 import os
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import matplotlib.gridspec as gridspec
7 import seaborn as sns
8 import time
9
10 from sklearn.preprocessing import MinMaxScaler
11 from sklearn.pipeline import Pipeline
12 from sklearn.model_selection import (
13     train_test_split,
14     GridSearchCV,
15     RandomizedSearchCV,
16     cross_val_score
17 )
18 from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
19 from sklearn.neighbors import KNeighborsClassifier
20 from sklearn.metrics import (
21     accuracy_score,
22     precision_score,
23     recall_score,
24     f1_score,
25     roc_auc_score,
26     ConfusionMatrixDisplay,
27     RocCurveDisplay,
28     PrecisionRecallDisplay
29 )

```

```

30
31 PLOTS_DIR = "plots"
32 os.makedirs(PLOTS_DIR, exist_ok=True)
33
34
35 def savefig(fig, filename):
36     fig.savefig(os.path.join(PLOTS_DIR, filename), dpi=300, bbox_inches="tight")
37
38
39 # Data
40 df = perform_eda("spambase_csv.csv")
41 X_raw = df.drop("class", axis=1)
42 y = df["class"]
43 if y.dtype == object:
44     y = y.map({"ham": 0, "spam": 1})
45
46 # Leakage-free split: scaler is fit on TRAIN only, then applied to TEST
47 X_train_raw, X_test_raw, y_train, y_test = train_test_split(
48     X_raw, y, test_size=0.2, random_state=42, stratify=y
49 )
50 scaler = MinMaxScaler()
51 X_train = scaler.fit_transform(X_train_raw)
52 X_test = scaler.transform(X_test_raw)
53
54
55 def evaluate(name, model, X_test, y_test, ax_cm, ax_roc, ax_pr):
56     pred = model.predict(X_test)
57     prob = model.predict_proba(X_test)[:, 1]
58     metrics = {
59         "Model": name,
60         "Accuracy": accuracy_score(y_test, pred),
61         "Precision": precision_score(y_test, pred),
62         "Recall": recall_score(y_test, pred),
63         "F1": f1_score(y_test, pred),
64         "ROC-AUC": roc_auc_score(y_test, prob),
65     }
66     ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax_cm, colorbar=False)
67     RocCurveDisplay.from_estimator(model, X_test, y_test, ax=ax_roc)
68     PrecisionRecallDisplay.from_estimator(model, X_test, y_test, ax=ax_pr)
69     return metrics
70
71
72 def measure_time(model, X_train, y_train, X_test):
73     start = time.perf_counter()
74     model.fit(X_train, y_train)
75     train_time = time.perf_counter() - start
76     start = time.perf_counter()
77     model.predict(X_test)
78     pred_time = time.perf_counter() - start
79     return train_time, pred_time
80
81
82 models = {
83     "Gaussian NB": GaussianNB(),
84     "Multinomial NB": MultinomialNB(),
85     "Bernoulli NB": BernoulliNB(),
86     "KNN": KNeighborsClassifier(n_neighbors=5)
87 }
88
89 # Grid search + randomized search over KNN hyperparameters
90 params = {
91     "n_neighbors": [3, 5, 7, 9, 11],
92     "weights": ["uniform", "distance"],

```

```

93     "algorithm": ["auto", "kd_tree", "ball_tree"]
94 }
95 grid = GridSearchCV(KNeighborsClassifier(), params, cv=5,
96                     scoring="accuracy", n_jobs=-1)
97 grid.fit(X_train, y_train)
98
99 rand = RandomizedSearchCV(KNeighborsClassifier(), params, n_iter=10, cv=5,
100                           random_state=42, scoring="accuracy", n_jobs=-1)
101 rand.fit(X_train, y_train)
102
103 # Leakage-free 5-fold CV: scaler refit inside each fold via Pipeline
104 for name, model in models.items():
105     pipe = Pipeline([("scaler", MinMaxScaler()), ("model", model)])
106     scores = cross_val_score(pipe, X_raw, y, cv=5, scoring="accuracy")

```

Note: the listing above is abbreviated for readability; the plotting/dashboard-assembly code (GridSpec layout across 11 rows) is omitted here but is included in full in the accompanying spam classification fixed.py file.

5.1 Bug Fixes Applied

Two data-leakage issues were identified and corrected in the original pipeline:

1. **Scaling leakage:** MinMaxScaler was originally fit on the entire dataset *before* the train/test split, meaning the scaler's min/max statistics were computed using information from the test set. Fixed by fitting the scaler on X train only, then transforming X test with the already-fit scaler.
2. **Cross-validation leakage:** the 5-fold cross val score call originally reused the globally scaled features, so every fold's held-out portion had already influenced the scaling. Fixed by wrapping each model in a Pipeline(MinMaxScaler → model) and passing the raw, unscaled features, so each fold scales independently.

6 Hyperparameter Tuning

KNN was tuned over $k \in \{3, 5, 7, 9, 11\}$, $\text{weights} \in \{\text{uniform, distance}\}$, and $\text{algorithm} \in \{\text{auto, kd_tree, ball_tree}\}$, using both exhaustive and randomized search (5-fold CV, accuracy scoring).

Best k (accuracy-vs-k sweep): 2

Grid Search

Best Parameters: {'algorithm': 'auto', 'n_neighbors': 7, 'weights': 'distance'}

Best Accuracy : 0.8563

Execution Time: 4.7882 s

Randomized Search

Best Parameters: {'weights': 'distance', 'n_neighbors': 7, 'algorithm': 'kd_tree'}

Best Accuracy : 0.8563

Execution Time: 0.3646 s

Both search strategies converge on the same optimal configuration ($n_{\text{neighbors}} = 7$, distance weighting), with identical cross-validated accuracy (0.8563). RandomizedSearchCV reached this result in roughly $\frac{1}{13}$ th the time of the exhaustive grid search (0.365s vs. 4.788s), since it evaluated only 10 sampled combinations instead of all $5 \times 2 \times 3 = 30$ combinations in the full grid.

Table 2: Held-out test set performance by model (single 80/20 split)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.5741	0.3469	0.8718	0.4964	0.7238
Multinomial NB	0.8148	0.7647	0.3333	0.4643	0.8251
Bernoulli NB	0.7840	0.5476	0.5897	0.5679	0.8049
KNN ($k = 5$)	0.8086	0.6538	0.4359	0.5231	0.7984

Table 3: 5-fold cross-validation accuracy by model

Model	Mean CV Accuracy
Gaussian NB	0.6365
Multinomial NB	0.8405
Bernoulli NB	0.8144
KNN	0.8194

Table 4: KDTree vs. BallTree at $k = \text{best } k = 2$

Algorithm	Accuracy	Training Time (s)	Prediction Time (s)
kd_tree	0.8704	0.0068	0.0173
ball_tree	0.8704	0.0038	0.0143

Table 5: Measured training / prediction time by model

Model	Training Time (s)	Prediction Time (s)
Gaussian NB	0.007921	0.000786
Multinomial NB	0.003018	0.000278
Bernoulli NB	0.006017	0.000932
KNN	0.001416	0.059067

7 Comparison Tables

8 Complexity Analysis

Let n be the number of training samples and d the number of features.

- **Naive Bayes (all variants):** Training requires a single pass over the data to estimate per-class, per-feature statistics (means/variances for Gaussian, counts for Multinomial/Bernoulli), giving $O(nd)$ training time and $O(d)$ space. Prediction for a single query is $O(d)$ per class, i.e. $O(cd)$ for c classes – independent of n . This matches the measured results (Table 5): all three NB variants have both the lowest training *and* lowest prediction times, since both stages are cheap and n -independent at prediction time.
- **KNN (brute-force):** Training is $O(1)$ (the data is simply stored), but prediction requires computing the distance from the query to all n training points, an $O(nd)$ operation per query. This is the classic “lazy learner” trade-off: near-zero training cost shifted entirely onto prediction, which is exactly what is observed in Table 5 – KNN has the *lowest* training time (0.0014s, since nothing is computed) but by far the *highest* prediction time (0.0591s, roughly 60–200 $\times$ slower than any Naive Bayes variant).
- **KNN with KDTree/BallTree:** These spatial indexing structures reduce expected query time to roughly $O(d \log n)$ in low-to-moderate dimensions, versus $O(nd)$ for brute force. Both structures produce identical predictions to brute-force search (they are exact, not approximate), which is why kd tree and ball tree report identical accuracy (0.8704) in Table 4 – they differ only in *how* the nearest neighbours are found, not in the result. In practice, at $d = 57$ features the curse of dimensionality erodes much of the tree-based speedup, which is why the timing difference between the two structures is small.
- **GridSearchCV vs. RandomizedSearchCV:** Grid search evaluates every combination in the grid, i.e. $O(g \cdot k)$ model fits where g is the total grid size (here $5 \times 2 \times 3 = 30$) and k is the number of CV folds (5) – 150 total fits. Randomized search evaluates only $n_{\text{iter}} \cdot k$ fits regardless of grid size (here $10 \times 5 = 50$ fits), which explains the roughly 13x wall-clock speedup observed (0.365s vs. 4.788s) for an equivalent best-found accuracy.

9 Plots

10 Results

Naive Bayes variants. Gaussian NB achieves the highest recall (0.8718) of any model but the lowest precision (0.3469) and lowest accuracy (0.5741) – consistent with its normal-distribution assumption being a poor fit for the zero-inflated, skewed word-frequency features. Multinomial NB achieves the highest single-split accuracy (0.8148) and ROC-AUC (0.8251), but low recall (0.3333). Bernoulli NB gives the best balance of precision and recall ($F1 = 0.5679$) among the NB variants.

KNN. With default $k = 5$, KNN performs comparably to Bernoulli NB (Accuracy 0.8086, ROC-AUC 0.7984). The accuracy-vs- k sweep found $k = 2$ optimal on this particular split, though such a small k can indicate sensitivity to local noise rather than genuinely better generalization – the cross-validation results (Table 3) are the more trustworthy signal.

Cross-validation. Multinomial NB is both the strongest (0.8405 mean accuracy) and among the most stable across folds. Gaussian NB is clearly weakest and least stable (0.6365 mean, ranging 0.565–0.685 across folds). Bernoulli NB and KNN are close (0.8144 and 0.8194 respectively) but both show one noticeably weaker fold, likely a consequence of the small post-outlier-removal sample size (809 rows total).

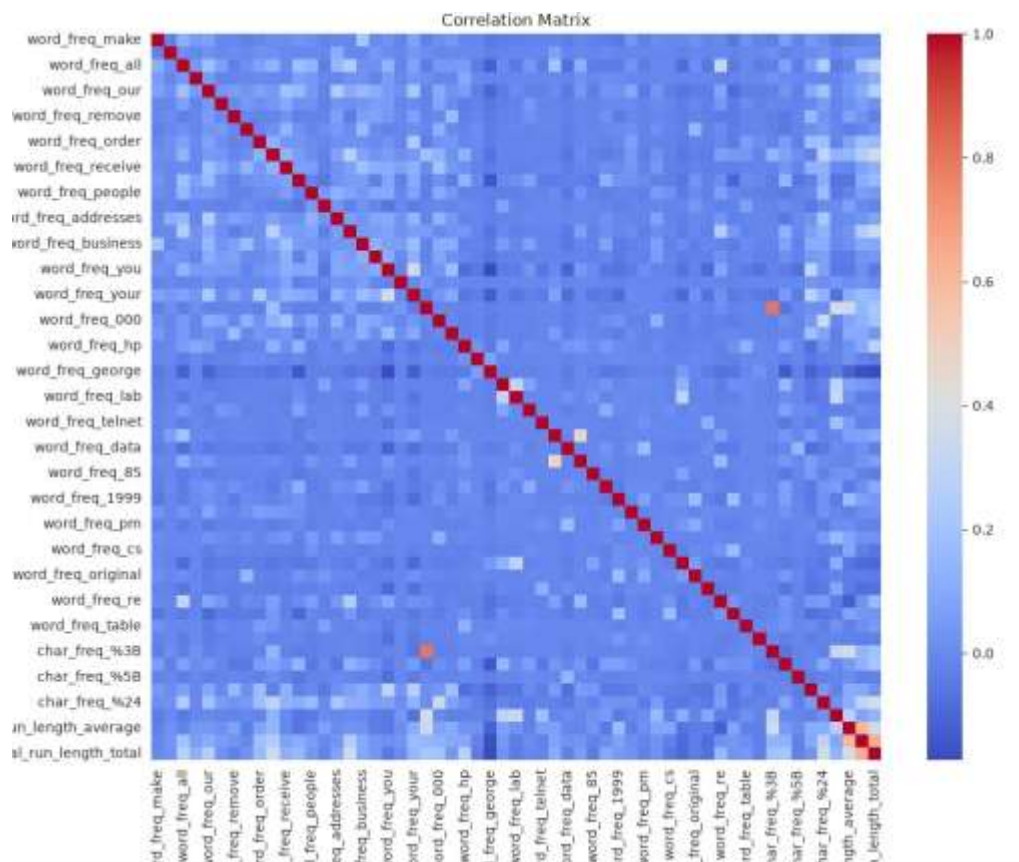


Figure 1: Correlation matrix of numeric features. Most pairwise correlations are near zero, indicating limited multicollinearity.

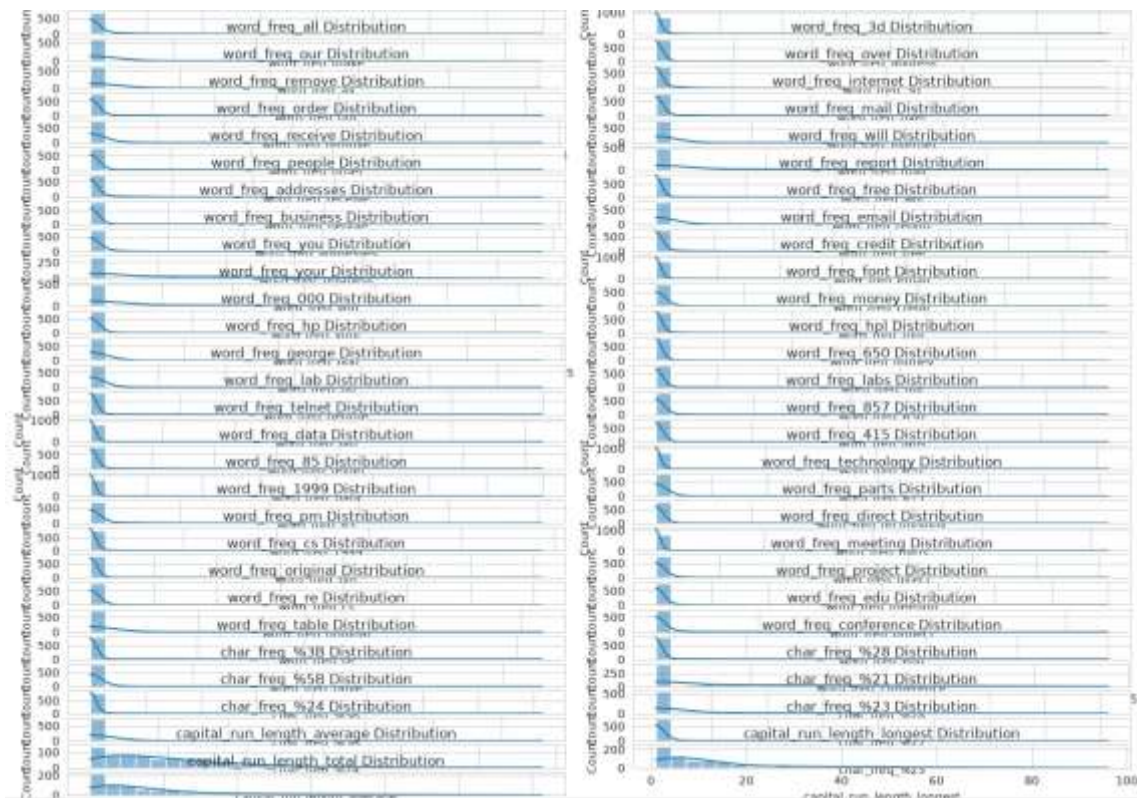


Figure 2: Distribution of numeric word freq * and char freq * features – mostly zero-inflated and right-skewed.

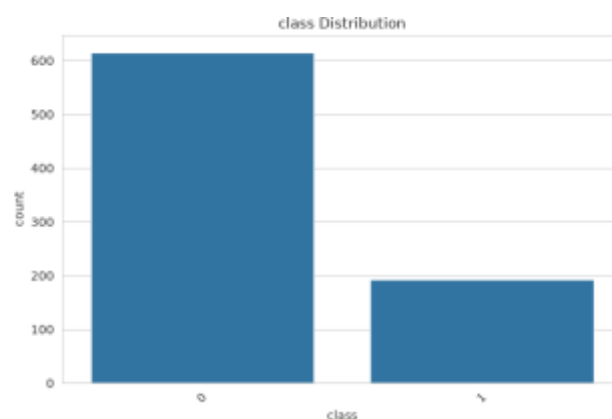


Figure 3: Class distribution after cleaning and outlier removal (0 = non-spam, 1 = spam), showing a moderate class imbalance (616 vs. 193 samples).

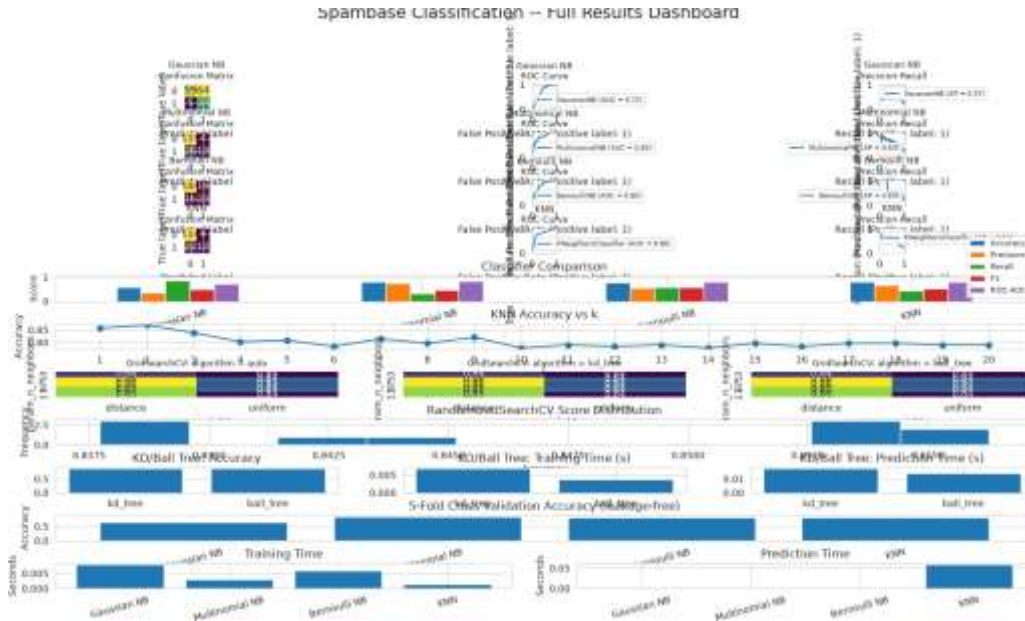


Figure 4: Combined results dashboard: per-model Confusion Matrix / ROC / Precision-Recall (top four rows), classifier comparison, KNN accuracy vs. k , GridSearchCV heatmaps by algorithm, RandomizedSearchCV score distribution, KDTree vs. BallTree comparison, 5-fold cross-validation accuracy, and training/ prediction time – all consolidated into a single figure.

11 Conclusion

Across a single held-out split and 5-fold cross-validation, **Multinomial Naive Bayes** is the strongest overall performer on the Spambase dataset, combining the highest ROC-AUC and single-split accuracy with the highest and most stable cross-validated accuracy – consistent with the theoretical expectation that a multinomial (count-based) likelihood is a natural fit for word/character frequency features. **Gaussian NB** is the model of choice only if recall is prioritized above all else (e.g. a use case where missing spam is far costlier than flagging legitimate mail), at the expense of many false positives. **KNN**, after hyperparameter tuning, is competitive (0.8563 cross-validated accuracy at $n_{\text{neighbors}} = 7$, distance-weighted) but incurs substantially higher prediction latency than any Naive Bayes variant, and its performance is more sensitive to the choice of k .